

คู่มืออธิบายโค้ดระบบ PFS สำหรับนำเสนออาจารย์

ตรวจสอบจากโค้ดในโครงการ PFS วันที่ 7 ตุลาคม 2569 คู่มือนี้เน้นฟังก์ชันหลักของหน้าออกแบบ Resume และส่วนที่เชื่อมกับการเข้าสู่ระบบ การบันทึก และการเผยแพร่ หมายเลขบรรทัดอ้างอิงโค้ด ณ วันที่ตรวจสอบ และอาจเปลี่ยนเมื่อแก้ไขไฟล์

บทพูดเปิดการนำเสนอ

ระบบนี้เป็นระบบสร้าง Resume และ Portfolio ครับ ผู้ใช้สามารถเพิ่มบล็อกข้อมูล แก้ไขข้อความ และรูปภาพ ลากจัดตำแหน่ง ปรับขนาดและรูปแบบ แล้วบันทึกหรือเผยแพร่ผ่านลิงก์ได้ ส่วนหน้าจอใช้ React บน Next.js และข้อมูลเก็บใน MongoDB ครับ

ภาพรวมการเชื่อมกัน

```
เปิดหน้า /student/editor
→ ตรวจสอบผู้ใช้และบทบาท
→ อ่านหรือสร้าง Portfolio จาก MongoDB
→ ส่งข้อมูลเข้า StudentEditor
→ ผู้ใช้แก้ไขข้อมูลและตำแหน่งใน React state
→ ส่ง PUT /api/portfolio/me
→ ตรวจสอบข้อมูลและบันทึกลง MongoDB
→ เมื่อเผยแพร่ เปิดดูได้ผ่าน /r/[slug] หรือ /p/[slug]
```

คำศัพท์ที่ใช้: **state** คือข้อมูลที่หน้าจอใช้อยู่, **API** คือช่องทางส่งข้อมูลระหว่างหน้าจอกับเซิร์ฟเวอร์, **section** คือบล็อกข้อมูลหนึ่งบล็อก, **frame** คือข้อมูลตำแหน่งและขนาดของบล็อก

แผนที่ไฟล์หลัก

ไฟล์	หน้าที่
app/student/editor/page.tsx	ตรวจสอบสิทธิ์และเตรียมข้อมูลก่อนเปิดหน้าออกแบบ
components/StudentEditor.tsx	รับการกระทำของผู้ใช้และจัดการข้อมูลหน้าออกแบบ
lib/resume-document.ts	คำนวณตำแหน่ง ขนาด พื้นที่ว่าง และเทมเพลต
components/resume/ResumeSection.tsx	แสดงข้อความ รูปภาพ และรูปแบบภายในบล็อก
components/ResumeRenderer.tsx	แสดง Resume ในหน้าตัวอย่างและหน้าสาธารณะ
app/resume-studio.css	รูปแบบหน้าจอ การเลื่อน และรูปแบบตอนพิมพ์
app/api/portfolio/me/route.ts	อ่านและบันทึก Portfolio
lib/portfolio.ts	เตรียมข้อมูล ตรวจสอบข้อมูล และจัดการ Portfolio
lib/mongodb.ts	เชื่อมต่อฐานข้อมูล

1. เข้าสู่ระบบ

เปิดโค้ดดังนี้:

- app/login/page.tsx บรรทัด 14 — รับข้อมูลจากฟอร์ม
- app/api/auth/login/route.ts บรรทัด 15 — ตรวจสอบบัญชีและสร้างเซสชัน
- lib/auth.ts บรรทัด 82 — ตรวจสอบรหัสผ่าน

พูดอธิบายได้เลย:

เมื่อกดเข้าสู่ระบบ หน้าจอจะส่งรหัสนักศึกษาหรืออีเมลและรหัสผ่านไปที่ API ครีป เซิร์ฟเวอร์ค้นหาบัญชีที่เปิดใช้งานใน MongoDB และตรวจสอบรหัสผ่าน ถ้าถูกต้องจะสร้างเซสชันในคุกกี้ แล้วหน้าจอจะไป Dashboard ครีป

ลำดับการทำงาน: ฟอร์ม → POST /api/auth/login → ตรวจสอบบัญชีและรหัสผ่าน → ตั้งคุกกี้ → Dashboard

หน้าจอไม่ได้ตรวจสอบรหัสผ่านกับฐานข้อมูลเอง การตรวจเกิดที่เซิร์ฟเวอร์ผ่าน `verifyPassword` โดยข้อมูลบัญชีอยู่ใน collection `users`

2. เปิดหน้าออกแบบและโหลดข้อมูล

เปิดโค้ดดังนี้:

- `app/student/editor/page.tsx` บรรทัด 10 — ตรวจสอบผู้ใช้และโหลดข้อมูล
- `lib/portfolio.ts` บรรทัด 555 — อ่านหรือสร้าง Portfolio
- `components/StudentEditor.tsx` บรรทัด 37 — เริ่มข้อมูลหน้าจอ

พูดอธิบายได้เลย:

ก่อนเปิดหน้าออกแบบ ระบบตรวจว่าผู้ใช้เข้าสู่ระบบและเป็นนักศึกษาครบ จากนั้นอ่าน Portfolio หรือสร้างให้ถ้ายังไม่มี แล้วส่งข้อมูลเข้า StudentEditor เพื่อแสดงบนหน้าจอ

ลำดับการทำงาน: ตรวจสอบผู้ใช้ → `getOrCreatePortfolio` → `serializePortfolio` → `initialPortfolio` → React state

`preparePortfolio` เตรียมตำแหน่งบล็อก และรองรับการแปลงข้อมูลการจัดวางแบบเก่าให้เป็น frame ที่หน้าออกแบบปัจจุบันใช้

3. เลื่อนหน้าหรือแผงขึ้นลง

เปิดโค้ดดังนี้:

- app/resume-studio.css บรรทัด 55 — เลื่อนรายการฝังซ้าย
- app/resume-studio.css บรรทัด 74 — เลื่อนพื้นที่กระดาษ
- app/resume-studio.css บรรทัด 140 — เลื่อนแผงปรับแต่งฝั่งขวา

พูดอธิบายได้เลย:

ส่วนที่เลื่อนขึ้นลงได้ใช้ CSS ครับ เรากำหนดพื้นที่ของแผงและใช้ `overflow: auto` เมื่อเนื้อหา
มากกว่าพื้นที่ที่กำหนด เบราวเซอร์จะให้เลื่อนดูได้โดยอัตโนมัติ ไม่ต้องเขียนฟังก์ชัน JavaScript รับ
การเลื่อนเองครับ

ลำดับการทำงาน: เนื้อหาเกินพื้นที่ → `overflow: auto` → เบราวเซอร์เปิดให้เลื่อน

`max-height: calc(100vh - 220px)` จำกัดความสูงของแผงตามหน้าจอ ส่วน `.rs-stage` เป็นพื้นที่รอบ
กระดาษ คำว่าเลื่อนแผงกับลากย้ายบล็อกเป็นคอนเซ็ปต์การทำงาน ส่วน `.rs-block-scroll` แม้ชื่อมี `scroll` แต่โค้ด
ปัจจุบันใช้ `overflow: hidden` จึงไม่ใช่จุดเปิดการเลื่อนภายในบล็อก

โค้ดสำคัญ (ตัดมาเฉพาะส่วน):

```
.rs-assets, .rs-layers {  
  max-height: calc(100vh - 220px);  
  min-height: 390px;  
  overflow: auto;  
  padding: 12px;  
}
```

4. เพิ่มบล็อกข้อมูล

เปิดโค้ดดังนี้:

- components/StudentEditor.tsx บรรทัด 98 — เพิ่มบล็อก
- lib/resume-document.ts บรรทัด 63 — หาพื้นที่ว่าง
- lib/resume-document.ts บรรทัด 115 — สร้างข้อมูลบล็อก

พูดอธิบายได้เลย:

เมื่อเพิ่มบล็อก ระบบกำหนดขนาดตามประเภทและหาพื้นที่ว่างบนกระดาษครับ จากนั้นสร้างบล็อกที่มีรหัสเฉพาะ เพิ่มเข้า sections และเลือกบล็อกใหม่ให้แก้ไขได้ทันที ถ้าไม่มีพื้นที่ว่างจะแจ้งว่ากระดาษเต็มครับ

ลำดับการทำงาน: เลือกองค์ประกอบ → addAsset → findFreeFrame → makeSection → markChanged

`makeSection` ใช้ `crypto.randomUUID()` สร้างรหัสบล็อก เมื่อใช้การลากจากรายการมาวางบนกระดาษ

`dropAsset` จะอ่านตำแหน่งเมาส์และหารด้วยค่า `zoom` เพื่อแปลงเป็นตำแหน่งบนกระดาษ

5. ลากย้ายบล็อก

เปิดโค้ดดังนี้:

- components/StudentEditor.tsx บรรทัด 461 — ส่วนประกอบลากและปรับขนาด
- components/StudentEditor.tsx บรรทัด 469 — รับตำแหน่งเมื่อปล่อยเมาส์
- components/StudentEditor.tsx บรรทัด 94 — แก่ข้อมูลตำแหน่ง
- lib/resume-document.ts บรรทัด 14 — จำกัดตำแหน่งให้อยู่บนกระดาษ

พูดอธิบายได้เลย:

การลากบล็อกใช้ไลบรารี react-rnd ครับ เมื่อปล่อยเมาส์ onDragStop จะรับค่า X และ Y ใหม่ แล้วส่งให้ updateFrame เพื่อแก้ไขข้อมูลของบล็อก ระบบจัดตำแหน่งตามกริดเมื่อเปิดตัวเลือกนี้ และจำกัดไม่ให้บล็อกออกนอกกระดาษครับ

ลำดับการทำงาน: ลาก → onDragStop → snap → updateFrame → normalizeFrame → เปลี่ยน state → แสดงตำแหน่งใหม่

`bounds="parent"` กำหนดขอบเขตการลาก, `position` รับตำแหน่งจาก frame, `scale={zoom}` ทำให้การลากสัมพันธ์กับการซูม และ `disableDragging={frame.locked}` ห้ามลากบล็อกที่ล็อกไว้ กริดที่เปิดใช้มีระยะ 8 พิกเซล

โค้ดสำคัญ (ตัดมาเฉพาะส่วน):

```
updateFrame(section.id, {
  x: snap(data.x, snapEnabled),
  y: snap(data.y, snapEnabled)
});
```

6. ปรับขนาดบล็อก

เปิดโค้ดดังนี้:

- components/StudentEditor.tsx บรรทัด 475 — รับขนาดหลังปรับ
- lib/resume-document.ts บรรทัด 14 — ตรวจสอบและจำกัดขนาด

พูดอธิบายได้เลย:

การปรับขนาดใช้ react-rnd เช่นเดียวกับการลากครีป หลังผู้ใช้ปล่อยจุดปรับขนาด ระบบอ่านความกว้างและความสูงใหม่ รวมถึงตำแหน่งที่เปลี่ยน แล้วบันทึกไว้ใน frame เพื่อให้หน้าจอแสดงขนาดใหม่

ลำดับการทำงาน: ยึดหรือหดยบล็อก → onResizeStop → อ่าน offsetWidth/offsetHeight → updateFrame
ปรับขนาดได้เมื่อบล็อกถูกเลือกและไม่ได้ล็อก ขนาดขั้นต่ำคือกว้าง 120 และสูง 72 พิกเซล สามารถกรอก X, Y, WIDTH และ HEIGHT ในแผงตำแหน่งได้ด้วย โดยเรียก `updateFrame` เช่นกัน

7. แก้ไขข้อความ สี ฟอนต์ และรูปแบบ

เปิดโค้ดดังนี้:

- components/StudentEditor.tsx บรรทัด 85 — แก้เฉพาะบล็อกที่เลือก
- components/StudentEditor.tsx บรรทัด 90 — แก้ค่ารูปแบบ
- components/StudentEditor.tsx บรรทัด 515 — ช่องแก้ไขข้อความ
- components/resume/ResumeSection.tsx บรรทัด 21 — นำค่ารูปแบบมาแสดง

พูดอธิบายได้เลย:

เมื่อพิมพ์ข้อความหรือเลือกสีและฟอนต์ ช่องกรอกจะเรียก onChange เพื่อแก้ไขข้อมูลของบล็อกที่เลือก
ครับ updateSection ค้นหาบล็อกตามรหัส ส่วน updateSettings ใช้แก้ค่ารูปแบบ แล้ว
ResumeSection นำข้อมูลใหม่ไปแสดงทันที

ลำดับการทำงาน: onChange → updateSection หรือ updateSettings → markChanged →
ResumeSection

ข้อความหลักเก็บใน `content.body` รายการเก็บใน `content.items` โดยแยกหนึ่งบรรทัดเป็นหนึ่งรายการ รูปแบบเก็บใน `settings` เช่น `fontFamily`, `fontSize` และ `alignment` ส่วนสีหัวข้อเก็บใน `accentColor`; ถ้าไม่ได้กำหนดฟอนต์เฉพาะบล็อกจะใช้ค่าหลักของ Resume

8. เพิ่มรูปภาพ

เปิดโค้ดดังนี้:

- components/StudentEditor.tsx บรรทัด 261 — ตรวจสอบไฟล์และอ่านรูป
- components/resume/ResumeSection.tsx บรรทัด 49 — แสดงรูปภาพ
- lib/portfolio.ts บรรทัด 639 — ตรวจสอบข้อมูลรูปที่เซิร์ฟเวอร์

พูดอธิบายได้เลย:

เมื่อเลือกไฟล์รูป ระบบตรวจสอบชนิดและขนาดก่อนครับ จากนั้นใช้ FileReader อ่านเป็น Data URL และเก็บใน imageUrl ของบล็อก เมื่อบันทึก Portfolio ข้อมูลรูปนี้จะถูกส่งไปพร้อมเนื้อหา ส่วนหน้าจอใช้แท็ก img แสดงรูปครับ

ลำดับการทำงาน: เลือกไฟล์ → ตรวจสอบชนิดและขนาด → readAsDataURL → content.imageUrl → แสดงรูป → บันทึกพร้อม Portfolio

รองรับ JPG, PNG, WebP และ GIF ขนาดไม่เกิน 900 KB วิธีนี้ไม่ได้ส่งไฟล์ไปยังบริการเก็บรูปแยกต่างหาก นอกจากนี้ยังใส่ URL รูปได้ และเลือก `cover` ให้เต็มพื้นที่หรือ `contain` ให้เห็นภาพทั้งหมด

9. ลบและทำสำเนาบล็อก

เปิดโค้ดตรงนี้:

- components/StudentEditor.tsx บรรทัด 127 — ลบบล็อก
- components/StudentEditor.tsx บรรทัด 113 — สร้างสำเนา

พูดอธิบายได้เลย:

การลบใช้ filter ตัดบล็อกที่มีรหัสตรงกับบล็อกที่เลือกออกจาก sections ครับ ส่วนการทำสำเนาจะคัดลอกข้อมูล สร้างรหัสใหม่ และเลื่อนตำแหน่งออกเล็กน้อยเพื่อแยกจากต้นฉบับ ทั้งสองอย่างอัปเดตข้อมูลผ่าน markChanged ครับ

ลำดับการทำงาน: ลบ → filter ตาม id; สำเนา → สร้าง id ใหม่ → เพิ่มเข้า sections

บล็อกที่ลบล้างไม่ได้ ส่วนสำเนาจะถูกปลดบล็อกและวางเหนือชั้นเดิม การลบรูปในบล็อกเป็นอีกคำสั่งหนึ่ง ซึ่งล้าง imageUrl โดยไม่ลบบล็อก

10. ชั้นของบล็อก การล็อก และการซ่อน

เปิดโค้ดตรงนี้:

- components/StudentEditor.tsx บรรทัด 135 — เปลี่ยนลำดับชั้น
- components/StudentEditor.tsx บรรทัด 484 — ใช้สถานะล็อก
- components/StudentEditor.tsx บรรทัด 512 — เปิดหรือปิดการแสดงผลบล็อก
- components/ResumeRenderer.tsx บรรทัด 55 — แสดงเฉพาะบล็อกที่เปิดไว้

พูดอธิบายได้เลย:

เมื่อบล็อกทับกัน ระบบใช้ zIndex กำหนดว่าบล็อกใดอยู่ด้านหน้าครับ ปุ่มนำไปหน้าหรือส่งไปหลัง เปลี่ยนลำดับชั้น ส่วนล็อกตำแหน่งใช้ป้องกันการลากและปรับขนาด และการซ่อนใช้ค่า visible เพื่อไม่แสดงบล็อกโดยยังเก็บข้อมูลไว้ครับ

ลำดับการทำงาน: นำไปหน้า/หลัง → moveLayer → zIndex; ล็อก → frame.locked; ซ่อน → visible

การล็อกเป็นการล็อกตำแหน่งและป้องกันการลบในคำสั่งที่ตรวจ locked ไม่ได้ปิดการแก้ไขเนื้อหาทั้งหมด การซ่อนต่างจากลบ เพราะข้อมูลยังอยู่ใน sections

11. ย้อนกลับและทำซ้ำ

เปิดโค้ดตรงนี้:

- components/StudentEditor.tsx บรรทัด 68 — เก็บประวัติการเปลี่ยนแปลง
- components/StudentEditor.tsx บรรทัด 145 — ย้อนกลับ
- components/StudentEditor.tsx บรรทัด 158 — ทำซ้ำ

พูดอธิบายได้เลย:

ระบบเก็บข้อมูลก่อนแก้ไขไว้ในประวัติครับ เมื่อกด Undo จะนำข้อมูลก่อนหน้ากลับมา และเก็บข้อมูลที่เพิ่งย้อนออกไว้สำหรับ Redo พอผู้ใช้แก้ไขใหม่ ระบบจะล้างเส้นทาง Redo และสร้างประวัติใหม่ครับ

ลำดับการทำงาน: แก้ไข → เก็บใน pastRef; Undo → ย้ายปัจจุบันไป futureRef; Redo → นำ future กลับมา

`markChanged` จำกัดรายการอดีตที่เก็บไว้ได้สูงสุด 50 รายการ และรวมการแก้ไขกลุ่มเดียวกันที่เกิดติดกันภายใน 800 มิลลิวินาที เพื่อลดการแยกประวัติทุกตัวอักษร ประวัติอยู่ในหน่วยความจำของหน้า ไม่ได้บันทึกเป็นประวัติถาวรในฐานข้อมูล

12. เลือกเทมเพลต

เปิดโค้ดดังนี้:

- components/StudentEditor.tsx บรรทัด 254 — รับเทมเพลตที่เลือก
- lib/resume-document.ts บรรทัด 100 — จัดวางบล็อกใหม่
- lib/resume-options.ts บรรทัด 13 — ตัวเลือกหลักบนหน้า

พูดอธิบายได้เลย:

เมื่อเลือกเทมเพลต ระบบนำบล็อกเดิมมาจัดตำแหน่งใหม่ตามรูปแบบที่กำหนดครับ จึงยังเก็บเนื้อหาเดิมไว้ หากเลือกเปลี่ยนสีด้วยก็จะเปลี่ยนสีหลักตามเทมเพลต ถ้าจัดวางไม่ได้เพราะพื้นที่ไม่พอ ระบบจะแจ้งและไม่ใช้ผลลัพธ์นั้นครับ

ลำดับการทำงาน: เลือกเทมเพลต → applyTemplate → applyDocumentTemplate → markChanged

หน้าออกแบบปัจจุบันแสดงตัวเลือกหลัก Professional, Modern และ Minimal ส่วนรหัสเทมเพลตอื่นยังมีการรองรับในโครงสร้างข้อมูล จึงควรอธิบายตามตัวเลือกที่หน้าจอปัจจุบันแสดง

13. ชุมกระดาศและปรับให้พอดีหน้าจอ

เปิดโค้ดตรงนี้:

- components/StudentEditor.tsx บรรทัด 276 — คำนวณขนาดพอดีพื้นที่
- components/StudentEditor.tsx บรรทัด 454 — ใช้ค่าชุมกับกระดาศ

พูดอธิบายได้เลย:

การชุมใช้ CSS transform: scale ตามค่า zoom ครับ ส่วนปุ่มพอดีหน้าจอจะคำนวณจากความกว้างพื้นที่ทำงาน และใช้ ResizeObserver ตรวจเมื่อพื้นที่เปลี่ยน การชุมเปลี่ยนขนาดที่เห็น แต่ไม่ได้เปลี่ยนค่าขนาดจริงของบล็อกครับ

ลำดับการทำงาน: กดชุมหรือพอดี → เปลี่ยน zoom → transform: scale

ขนาดกระดาศในข้อมูลคือ 794×1123 พิกเซล ตาม `documentSize` การลากยังใช้ตำแหน่งในหน่วยกระดาศ โดยส่ง zoom ให้ react-rnd และหาร zoom เมื่อวางองค์ประกอบจากรายการ

14. บันทึกและบันทึกอัตโนมัติ

เปิดโค้ดตรงนี้:

- components/StudentEditor.tsx บรรทัด 171 — ส่งข้อมูลไป API
- components/StudentEditor.tsx บรรทัด 211 — ตั้งเวลาบันทึกอัตโนมัติ
- app/api/portfolio/me/route.ts บรรทัด 35 — ตรวจสอบข้อมูลและเขียน MongoDB
- lib/portfolio.ts บรรทัด 639 — ตรวจสอบความถูกต้อง

พูดอธิบายได้เลย:

เมื่อมีการแก้ไข ระบบเปลี่ยนสถานะเป็นยังไม่บันทึก และตั้งเวลาบันทึกหลังการแก้ไขล่าสุดประมาณ 1.1 วินาทีครับ ฟังก์ชัน save ส่งข้อมูล Portfolio ไปที่ API ด้วย PUT ฟังก์ชันเช็คว่าตรวจสอบสิทธิ์และข้อมูล แล้วอัปเดต collection portfolios ใน MongoDB ครับ

ลำดับการทำงาน: markChanged → revision เพิ่ม → รอ 1100 ms → save → PUT → ตรวจสอบและปรับข้อมูล → updateOne → แสดงบันทึกแล้ว

ผู้ใช้งานบันทึกเองได้ด้วย ระบบใช้เลข revision แยกข้อมูลที่ส่งไปแล้วกับข้อมูลที่แก้ไขเพิ่ม และใช้ savePromiseRef ป้องกันการเริ่มคำขอบันทึกซ้อนกัน ถ้าเชื่อมต่อไม่ได้จะแจ้งข้อผิดพลาด โดยข้อมูลยังอยู่ในหน้าที่เปิดอยู่ ไม่ใช่การเก็บสำรองถาวรแบบออฟไลน์

โค้ดสำคัญ (ตัดมาเฉพาะส่วน):

```
const response = await fetch("/api/portfolio/me", {
  method: "PUT",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(snapshot)
});
```

15. เผยแพร่และปิดเผยแพร่

เปิดโค้ดตรงนี้:

- components/StudentEditor.tsx บรรทัด 223 — บันทึกก่อนเผยแพร่
- app/api/portfolio/me/publish/route.ts บรรทัด 15 — เปลี่ยนสถานะเป็น published
- app/api/portfolio/me/unpublish/route.ts บรรทัด 15 — เปลี่ยนสถานะเป็น unpublished
- [app/p/[slug]/page.tsx บรรทัด 13](/Users/surachartlimrattanaphun/Desktop/PFS/app/p/[slug]/page.tsx:13) — อ่านเฉพาะผลงานเผยแพร่แล้ว

พูดอธิบายได้เลย:

ก่อนเผยแพร่ ระบบบันทึกการแก้ไขให้สำเร็จก่อนครับ แล้วเรียก API เปลี่ยนสถานะเป็น published หน้าสาธารณะจะค้นหาผลงานจาก slug และต้องมีสถานะ published จึงเปิดดูได้ เมื่อปิดเผยแพร่ ข้อมูลยังอยู่ แต่หน้าสาธารณะจะไม่พบผลงานนั้นครับ

ลำดับการทำงาน: save → POST publish → status published → เปิดลิงก์; ปิดเผยแพร่ → status unpublished

`slug` คือชื่อที่ใช้ใน URL; `/r/[slug]` ส่งต่อการทำงานให้หน้า `/p/[slug]` ซึ่งใช้ `ResumeRenderer` แสดงผล API เผยแพร่มีการตรวจ slug ซ้ำด้วย การบันทึกแก้ไขผลงานที่เผยแพร่แล้วจะยังคงสถานะ published จึงไม่ได้แยกฉบับร่างใหม่จากฉบับสาธารณะ

16. ดูตัวอย่างและบันทึกเป็น PDF

เปิดโค้ดดังนี้:

- components/StudentEditor.tsx บรรทัด 352 — เตรียมหน้าก่อนพิมพ์
- components/ResumeRenderer.tsx บรรทัด 20 — แสดง Resume
- app/resume-studio.css บรรทัด 197 — กำหนดรูปแบบพิมพ์ A4

พูดอธิบายได้เลย:

หน้าตัวอย่างใช้ ResumeRenderer แสดงข้อมูลเกี่ยวกับหน้าออกแบบครับ เมื่อกดบันทึก PDF ระบบตรวจเนื้อหาแล้ว เปิดตัวอย่าง และรอให้พอนด์กับรูปพร้อม จากนั้นเรียก window.print เพื่อเปิดหน้าต่างพิมพ์ของเบราว์เซอร์ ผู้ใช้เลือกบันทึกเป็น PDF ได้ครับ

ลำดับการทำงาน: ตรวจเนื้อหาแล้ว → เปิด preview → รอพอนด์และรูป → window.print → เลือก Save as PDF

CSS ตอนพิมพ์กำหนด A4 แนวตั้งและซ่อนแถบเครื่องมือ วิธีนี้ใช้การพิมพ์ของเบราว์เซอร์ ไม่ได้สร้างไฟล์ PDF ที่เซิร์ฟเวอร์โดยตรง

17. แจ้งเตือนเนื้อหาล้นบล็อก

เปิดโค้ดดังนี้:

- components/resume/ResumeSection.tsx บรรทัด 34 — เปรียบเทียบพื้นที่เนื้อหากับบล็อก
- components/StudentEditor.tsx บรรทัด 314 — เก็บรายการบล็อกที่ล้น
- components/StudentEditor.tsx บรรทัด 353 — หยุดการพิมพ์เมื่อพบเนื้อหาล้น

พูดอธิบายได้เลย:

ระบบตรวจสอบว่าความสูงหรือความกว้างของเนื้อหามากกว่าพื้นที่บล็อกหรือไม่ครับ โดยเปรียบเทียบ scrollHeight กับ clientHeight และ scrollWidth กับ clientWidth ถ้าพบว่าล้นจะรายงานรหัสบล็อกให้หน้าออกแบบแสดงคำเตือน และให้ผู้ใช้ขยายบล็อกก่อนพิมพ์ครับ

ลำดับการทำงาน: ResumeSection วัดพื้นที่ → reportOverflow → overflowIds → แจ้งเตือน

ใช้ ResizeObserver ตรวจสอบเมื่อพื้นที่เปลี่ยน มีค่าพออนผัน 2 พิกเซลในการเปรียบเทียบ การตรวจนี้ช่วยระบุว่าพื้นที่ไม่พอ ไม่ได้ย่อพอนต์หรือขยายบล็อกให้อัตโนมัติ

บทสรุปการเชื่อมต่อ

แม้หน้าจะเกี่ยวข้องกับหลายไฟล์ แต่แบ่งหน้าที่ได้รับ StudentEditor รับการกระทำและอัปเดตข้อมูล, resume-document ช่วยคำนวณการจัดวาง, ResumeSection แสดงเนื้อหาภายในบล็อก และ API รับข้อมูลไปบันทึก MongoDB ส่วน CSS จัดหน้าตาและการเลื่อนครับ

คำถามที่อาจารย์อาจถาม

คำถาม	คำตอบสั้น ๆ
ทำไมแก้แล้วหน้าจอเปลี่ยนทันที?	เพราะแก้ React state แล้ว React แสดงผลใหม่ โดยยังไม่ต้องรอฐานข้อมูล
ลากได้เพราะโค้ดตรงไหน?	<code><Rnd></code> ใน StudentEditor และ <code>onDragStop</code> ที่เรียก <code>updateFrame</code>
เลื่อนแพนได้เพราะอะไร?	CSS <code>overflow: auto</code> ร่วมกับการกำหนดพื้นที่ของแพน
ตำแหน่งเก็บไว้ที่ไหน?	ใน frame ของแต่ละ section ได้แก่ x, y, width, height และ zIndex แล้วบันทึกพร้อม Portfolio
ใช้กริด 12 คอลัมน์ในการลากปัจจุบันไหม?	หน้าออกแบบปัจจุบันใช้ frame และ react-rnd มีการจัดแนวทุก 8 พิกเซล ส่วนข้อมูลกริดเก่ามีฟังก์ชันแปลงให้
ข้อมูลเก็บฐานข้อมูลอะไร?	MongoDB โดยข้อมูลผลงานอยู่ใน collection portfolios และบัญชีอยู่ใน users
ซ่อนกับลบต่างกันอย่างไร?	ซ่อนเปลี่ยน visible แต่เก็บข้อมูลไว้ ลบเอาบล็อกออกจาก sections
ล็อกแล้วแก้ข้อความได้ไหม?	ได้ การล็อกใช้ควบคุมตำแหน่ง การปรับขนาด และการลบที่ตรวจสอบสถานะล็อก
ปิดเว็บแล้ว Undo ยังอยู่ไหม?	ประวัติ Undo/Redo อยู่ในหน่วยความจำของหน้าที่เปิด ไม่ใช่ประวัติถาวรในฐานข้อมูล
เผยแพร่คืออะไร?	เปลี่ยนสถานะเป็น published เพื่อให้หน้าสาธารณะค้นหาและแสดงผลงานได้
PDF สร้างด้วยอะไร?	ใช้ window.print และ CSS สำหรับพิมพ์ ผู้ใช้เลือกบันทึก PDF ในเบราว์เซอร์

ลำดับสถิติที่แนะนำ

1. เข้าสู่ระบบและเปิดหน้าออกแบบ อธิบายการโหลดข้อมูล
2. เลื่อนแผงซ้ายหรือขวา ซี CSS overflow: auto
3. เพิ่มบล็อก แล้วลากและปรับขนาด ซี addAsset, Rnd และ updateFrame
4. แก้ข้อความ สี และฟอนต์ ซี updateSection, updateSettings และ ResumeSection
5. ทำสำเนา ลบ และ Undo เพื่ออธิบายข้อมูล sections กับประวัติ
6. รอสถานะบันทึกแล้ว อธิบายเส้นทางจาก save ไป MongoDB
7. ดูตัวอย่างและเปิดหน้าต่างพิมพ์เพื่อบันทึก PDF
8. หากพร้อมให้ผลงานเปิดดูสาธารณะ จึงสถิติเผยแพร่และเปิดลิงก์

เอกสารนี้อธิบายจากการอ่านโค้ดจริง ไม่ใช่รายงานทดสอบการใช้งานทุกฟังก์ชัน และไม่ครอบคลุมรายละเอียดทุกหน้าของผู้ดูแลระบบ